{ Python }

# Python for Odoo Development

## بايثون لتطوير أودو

Complete Python Course — From Zero to Odoo

كورس بايثون الشامل — من الصفر حتى أودو

**Python 3** **Odoo Ready** **Beginner**

# Course Outline

01 **Introduction to Python**
مقدمة إلى بايثون

02 **Variables & Data Types**
المتغيرات وأنواع البيانات

03 **Operators**
العوامل

04 **Control Flow**
التحكم في التدفق

05 **Data Structures**
هياكل البيانات

06 **Strings**
النصوص

07 **Functions**
الدوال

08 **Error Handling**
معالجة الأخطاء

09 **Object-Oriented Programming**
البرمجة كائنية التوجه

10 **Practical Projects**
مشاريع عملية

01

# Introduction to Python

مقدمة إلى بايثون

5 Lessons



# 01 What is Python?

**English**

- High-level, general-purpose programming language
- Created by Guido van Rossum in 1991
- Readable, clean syntax — beginner friendly
- Cross-platform: Windows, Mac, Linux
- Used in web, data science, AI, automation & Odoo

**العربية**

- لغة برمجة عالية المستوى وعامة الاستخدام
- أنشأها جيدو فان روسوم عام ١٩٩١
- صياغة بسيطة وواضحة — مناسبة للمبتدئين
- تعمل على ويندوز وماك ولينكس
- تُستخدم في الويب والبيانات والذكاء الاصطناعي وأودو

★ **Key Fact:** Python is the #1 most popular programming language worldwide (TIOBE 2024)
بايثون هي لغة البرمجة الأكثر شيوعاً في العالم وفق مؤشر TIOBE لعام ٢٠٢٤

# 02 Where Python is Used

### Web Development
تطوير الويب

Django, Flask, FastAPI

### Data Science
علم البيانات

Pandas, NumPy, Matplotlib

### AI & ML
الذكاء الاصطناعي

TensorFlow, PyTorch

### Automation
الأتمتة

Scripts, file management

### ERP — Odoo
أودو ERP

Modules, ORM, business logic

### Scientific
الحوسبة العلمية

SciPy, SymPy, research

# 03 Installing Python & IDEs

## Installation Steps | خطوات التثبيت

1. **Open python.org**
   افتح python.org
2. **Download Python 3.x**
   حمّل بايثون 3.x
3. **Run installer → Add to PATH**
   شغّل وفعّل Add to PATH
4. **Terminal: python --version**
   الطرفية: python --version
5. **Install your IDE**
   ثبّت بيئة التطوير

## Choose Your IDE | اختر بيئتك

★ Recommended

### VS Code
Lightweight & powerful
خفيف وسريع ومتعدد الاستخدامات

### PyCharm
Full-featured Python IDE
بيئة متكاملة ومتخصصة

### Jupyter Notebook
Best for data & analysis
مثالي للبيانات والتحليل

# 04 Running Python (REPL & Scripts)

## REPL — Interactive Mode

- Run code line-by-line interactively
- Type: python in your terminal
- Press Enter to execute each line
- Perfect for quick tests

```python
>>> python
>>> 2 + 3
5
>>> name = 'Ahmed'
>>> print(f'Hi {name}')
Hi Ahmed
>>> type(name)
<class 'str'>
```

## Script Files — .py

- Write full programs saved as .py files
- Run with: python filename.py
- Best for real projects and apps
- Can use any text editor or IDE

```python
# main.py
name = 'Ahmed'
age = 25

def greet(n):
    print(f'Hello {n}!')

greet(name)
# Run: python main.py
```

# 05 Python Syntax Basics

```python
# Rule 1: Indentation (4 spaces)
if age >= 18:
    print('Adult')
    if age >= 65:
        print('Senior')

# Rule 2: Comments start with #
# This line is ignored by Python

# Rule 3: No semicolons needed
name = 'Ahmed'
print(name)

# Rule 4: Case sensitive
Name = 'Ali'  # ≠ name ≠ NAME
```

### Indentation matters!

المسافة البادئة إلزامية

4 spaces — never use tabs

### No semicolons needed

لا فاصلة منقوطة

Just press Enter to end a line

### Comments → start with #

# التعليقات تبدأ بـ

```
# This line is ignored by Python
```

### Case sensitive

حساسة لحالة الأحرف

Name ≠ name ≠ NAME

02

# Variables & Data Types

المتغيرات وأنواع البيانات

4 Lessons



# 06 Variables & Naming Rules

**English**

- Variables store data values in memory
- Must start with a letter or underscore ( _ )
- Can contain letters, numbers, underscores
- Case-sensitive: age ≠ Age ≠ AGE
- Cannot use Python reserved keywords

**العربية**

- المتغيرات تخزّن قيم البيانات في الذاكرة
- ( _ ) تبدأ بحرف أو شرطة سفلية
- تحتوي حروف وأرقام وشرطة سفلية فقط
- حساسة لحالة الأحرف: age ≠ Age ≠ AGE
- لا تستخدم الكلمات المحجوزة في بايثون

python

```python
name   = 'Ahmed'    #    valid string variable
_price = 99.5       #    valid (underscore start)
age2   = 30         #    valid (number inside ok)
2name  = 'Ali'      #    starts with number — ERROR
my-var = 10         #    hyphen not allowed — ERROR
for    = 5          #    reserved keyword — ERROR
```

# 07 Built-in Data Types

**int**
`age = 25`
Whole numbers | أعداد صحيحة

**float**
`price = 9.99`
Decimal numbers | أعداد عشرية

**str**
`name = "Ahmed"`
Text strings | نصوص

**bool**
`active = True`
True / False | صح أو خطأ

**list**
`x = [1, 2, 3]`
Ordered, mutable | قائمة قابلة للتعديل

**tuple**
`p = (1, 2)`
Ordered, immutable | قائمة ثابتة

**dict**
`d = {"k":"v"}`
Key-value pairs | قاموس

**set**
`s = {1, 2, 3}`
Unique values | قيم فريدة

Tip: Use type(x) to check any variable's type | استخدم type() للتحقق من نوع أي متغير

# 08 Type Conversion & Input/Output

```python
# Type Conversion (Casting)
x = int("42")        # "42" → 42
y = float("3.14")    # str  → 3.14
z = str(100)         # int  → "100"
b = bool(0)          # int  → False
b2= bool("hello")    # str  → True

# input() always returns str
name = input("Name: ")
age  = int(input("Age: "))
print(f"Hello {name}, {age+1}!")

# print() formatting
print(f"Pi = {3.14159:.2f}")
print(f"{1000:,}")
```

**English**

- int() → convert to integer
- float() → convert to decimal
- str() → convert to string
- bool() → convert to True/False
- input() always returns a string
- print() with f-strings for formatting

**العربية**

- int() ← تحويل إلى عدد صحيح
- float() ← تحويل إلى عشري
- str() ← تحويل إلى نص
- bool() ← تحويل إلى صح أو خطأ
- input() تُرجع نصاً دائماً
- print() مع f-strings للتنسيق

03

# Operators

العوامل

5 Lessons



# 10-12 Arithmetic, Comparison & Logical Operators

## Arithmetic | حسابية

| | |
|---|---|
| + | 5 + 3 = 8 |
| - | 10 - 4 = 6 |
| * | 3 * 4 = 12 |
| / | 10 / 3 = 3.33 |
| // | 10 // 3 = 3 |
| % | 10 % 3 = 1 |
| ** | 2 ** 8 = 256 |

## Comparison | مقارنة

| | |
|---|---|
| == | **Equal to** يساوي |
| != | **Not equal to** لا يساوي |
| > | **Greater than** أكبر من |
| < | **Less than** أصغر من |
| >= | **Greater or equal** أكبر أو يساوي |
| <= | **Less or equal** أصغر أو يساوي |

## Logical | منطقية

```python
and → both True
or  → one True
not → reverse

x>5 and x<10
x<0 or x>5
not True # False
```

## Assignment | تـعيين

```python
x = 10  # assign
x += 5  # x=15
x -= 3  # x=12
x *= 2  # x=24
x //= 5 # x=4
```

04

# Control Flow

التحكم في التدفق

3 Lessons



# 15-16 Conditional Statements & Nested Conditions

python

```python
score = 75

if score >= 90:
    print('Excellent!')
elif score >= 75:
    print('Good job!')
elif score >= 60:
    print('Pass')
else:
    print('Fail')
```

python

```python
# Nested conditions
age    = 20
has_id = True

if age >= 18:
    if has_id:
        print('Access OK')
    else:
        print('Show ID')
else:
    print('Too young')
```

# 17 Loops — for, while, break, continue

python

```python
# for loop
for i in range(5):
    print(i)  # 0 1 2 3 4

# loop over list
names = ['Ali','Sara','Mona']
for name in names:
    print(f'Hi {name}')

# break
for i in range(10):
    if i == 5: break
    print(i)  # 0 1 2 3 4
```

python

```python
# while loop
count = 0
while count < 5:
    print(count)
    count += 1

# continue
for i in range(6):
    if i % 2 == 0:
        continue
    print(i)  # 1 3 5

# pass (placeholder)
for i in range(3):
    pass  # do nothing
```

05

# Data Structures

هياكل البيانات

4 Lessons



# 18 Lists

```python
# Create
fruits = ['apple','banana','cherry']

# Index (starts at 0)
print(fruits[0])    # apple
print(fruits[-1])   # cherry

# Slice [start:stop]
print(fruits[0:2])  # ['apple','banana']

# Modify
fruits[0] = 'mango'
fruits.append('grape')
fruits.insert(1, 'kiwi')
fruits.remove('banana')
fruits.pop()        # remove last
```

## Key List Methods | أهم دوال القوائم

| | |
|---|---|
| `append(x)` | **Add x to end** — أضف x للنهاية |
| `insert(i,x)` | **Insert x at index i** — أدخل x عند الفهرس |
| `remove(x)` | **Remove first x** — احذف أول x |
| `pop(i)` | **Remove & return item** — احذف وأرجع العنصر |
| `sort()` | **Sort in place** — رتّب القائمة |
| `reverse()` | **Reverse in place** — اقلب الترتيب |
| `len(list)` | **Get number of items** — عدد العناصر |

# 19-21 Tuples, Sets & Dictionaries

## tuple

**Ordered, immutable list**

قائمة مرتبة ثابتة

```python
# Immutable list
p = (10, 20, 30)
print(p[0])  # 10
# p[0]=1  → ERROR
```

## set

**Unique values only**

قيم فريدة فقط

```python
# Unique values
s = {1,2,2,3,3}
print(s)  # {1,2,3}
s.add(4)
```

## dict

**Key-value pairs**

أزواج مفتاح وقيمة

```python
# Key-value
p={'name':'Ali'}
p['age']=25
print(p['name'])
```

```python
# Dictionary — Full Example
person = {'name':'Ahmed', 'age':25, 'city':'Cairo'}
person['email'] = 'a@x.com'          # add new key
print(person.keys())                  # dict_keys([...])
print(person.values())                # dict_values([...])
for key, val in person.items():       # loop all pairs
    print(f'{key}: {val}')
```

06

# Strings

النصوص

4 Lessons



# 22-25 String Indexing, Methods, Formatting & Escape

```python
# Indexing & Slicing
text = "Hello, World!"
print(text[0])     # H
print(text[-1])    # !
print(text[0:5])   # Hello
print(text[7:])    # World!
print(text[::-1])  # !dlroW ,olleH
print(len(text))   # 13

# String Methods
"hello".upper()    # HELLO
"  hi  ".strip()   # hi
"Hi".replace("H","h") # hi
"a,b".split(",")   # ["a","b"]

# f-string
name="Ahmed"; age=25
print(f"Hi {name}, age {age}")
print(f"Pi={3.14:.2f}")
```

## Escape Characters | محارف الهروب

| | |
|---|---|
| `\n` | **New line** سطر جديد |
| `\t` | **Tab space** مسافة جدول |
| `\'` | **Single quote** اقتباس مفرد |
| `\"` | **Double quote** اقتباس مزدوج |
| `\\` | **Backslash** شرطة عكسية |
| `\r` | **Carriage return** رجوع المؤشر |

07

# Functions

الحوال

6 Lessons



# 26-29 Defining Functions, Parameters, Return & Defaults

```python
# Define a function
def greet(name):
    print(f'Hello {name}!')

greet('Ahmed')                          # calling the function

# Return a value
def add(a, b):
    return a + b

result = add(3, 5)                      # result = 8

# Default arguments
def greet(name, msg='Welcome'):
    print(f'{msg}, {name}!')

greet('Ali')                            # Welcome, Ali!
greet('Sara', 'Hello')                  # Hello, Sara!

# Keyword arguments — any order
def info(name, age, city):
    print(f'{name}, {age}, {city}')
```

# 31 Scope — Local & Global

```python
# LOCAL scope
def my_func():
    x = 10      # local var
    print(x)    # works

my_func()        # 10
# print(x)       # ERROR

# GLOBAL scope
name = 'Ahmed'  # global var

def say_hi():
    print(name) # can read

# Modify global inside function
counter = 0
def increment():
    global counter
    counter += 1

increment()
print(counter)   # 1
```

**English**

- Local: variable defined inside a function
- Only accessible within that function
- Destroyed when function returns
- Parameters are always local variables
- global keyword: modify a global inside a function

**العربية**

- محلي: متغير معرّف داخل الدالة
- لا يمكن الوصول إليه من خارج الدالة
- يُحذف بعد انتهاء تنفيذ الدالة
- المعاملات دائماً متغيرات محلية
- تـــعـديل متغير عام من: global داخل الدالة

08

# Error Handling

معالجة الأخطاء

3 Lessons



# 32 Types of Errors

### SyntaxError

**Invalid syntax** صياغة غير صحيحة

```
e.g.  print("hi"
```

### TypeError

**Wrong type operation** عملية على نوع خاطئ

```
e.g.  '5' + 5
```

### NameError

**Variable not defined** متغير غير معرّف

```
e.g.  print(x)
```

### IndexError

**Index out of range** فهرس خارج النطاق

```
e.g.  [1,2,3][9]
```

### KeyError

**Dict key not found** مفتاح القاموس غير موجود

```
e.g.  {"a":1}["b"]
```

### ZeroDivisionError

**Division by zero** قسمة على صفر

```
e.g.  10 / 0
```

# 33-35 try, except, else, finally & raise

```python
try:
    x = int(input('Num: '))
    result = 10 / x
    print(f'Result: {result}')
except ValueError:
    print('Enter a valid number!')
except ZeroDivisionError:
    print('Cannot divide by zero!')
except Exception as e:
    print(f'Error: {e}')
else:
    print('No errors!')
finally:
    print('Always runs.')

# raise custom exception
def check_age(age):
    if age < 0:
        raise ValueError('Negative!')
    return age
```

**English**

- try → code block that might fail
- except → handles the specific error
- else → runs only if no error occurred
- finally → always runs (cleanup code)
- raise → throw your own exception

**العربية**

- try ← الكود الذي قد يتسبب في خطأ
- except ← يعالج نوعاً محدداً من الأخطاء
- else ← تعمل فقط إذا لم يحدث خطأ
- finally ← تعمل دائماً (تنظيف الكود)
- raise ← إطلاق استثناء مخصص

09

# Object-Oriented Programming

البرمجة كائنية التوجه

5 Lessons



# 36-38 Classes, __init__ & Attributes

```python
class Person:
    # Class attribute (shared)
    species = 'Human'

    def __init__(self, name, age):
        self.name = name  # instance
        self.age  = age   # instance

    def greet(self):
        print(f'Hi, I am {self.name}')

    def birthday(self):
        self.age += 1
        return self.age

# Create objects
p1 = Person('Ahmed', 25)
p2 = Person('Sara',  30)

p1.greet()              # Hi, I am Ahmed
print(p2.age)           # 30
print(Person.species)   # Human
```

**English**

- class → blueprint / template for objects
- __init__ → constructor, runs on creation
- self → refers to the current object instance
- Attributes → object's data (variables)
- Methods → object's behavior (functions)
- Objects are instances of the class

**العربية**

- class ← قالب لإنشاء الكائنات
- __init__ ← المنشئ يعمل عند الإنشاء
- self ← يشير إلى الكائن الحالي
- الخصائص ← بيانات الكائن (متغيرات)
- الدوال ← سلوك الكائن (functions)
- الكائنات هي نسخ من الكلاس

# 39-40 Inheritance & Encapsulation

```python
# Inheritance
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return 'Some sound'

class Dog(Animal):      # inherits
    def speak(self):    # override
        return 'Woof!'

class Cat(Animal):
    def speak(self):
        return 'Meow!'

d = Dog('Rex')
print(d.name)    # Rex
print(d.speak()) # Woof!

# Encapsulation
class BankAccount:
    def __init__(self, bal):
        self.__balance = bal  # private
```

**English**

- Inheritance: child gets parent features
- Child class can override parent methods
- super() calls the parent class method
- Encapsulation: hide data with __ prefix
- Use getter methods to access private data

**العربية**

- الوراثة: الابن يرث خصائص الأب
- الابن يمكنه تعديل دوال الأب (override)
- يستدعي كلاس الأب super()
- __ التغليف: إخفاء البيانات بـ
- استخدم getter للوصول للبيانات الخاصة

10

# Practical Projects

مشاريع عملية

4 Lessons



# Practical Projects Overview

**P1**

## Classes & Objects

Build a Student class with name, grade attributes and methods to calculate GPA and display info

كلاسات وكائنات

بناء كلاس Student بخصائص ودوال لحساب المعدل وعرض المعلومات

**P2**

## Calculator Program

Full CLI calculator (+, -, *, /) with a loop, error handling and formatted output

برنامج حاسبة

حاسبة سطر أوامر كاملة مع حلقة ومعالجة أخطاء ومخرجات منسقة

**P3**

## To-Do List App (CLI)

Add, view, complete and delete tasks with persistent file storage using open()

تطبيق قائمة المهام

إضافة وعرض وإكمال وحذف المهام مع حفظ دائم في ملف باستخدام open()

**P4**

## Number Guessing Game

Random number game with hints (higher/lower), attempt counter and score tracking

لعبة تخمين الرقم

لعبة رقم عشوائي مع تلميحات وعداد محاولات وتتبع النقاط

# Project — Calculator Program

python

```python
def calculator():
    while True:
        print('\n=== Calculator ===')
        print('1) Add  2) Sub  3) Mul  4) Div  5) Quit')
        choice = input('Choose (1-5): ')

        if choice == '5':
            print('Goodbye!'); break

        if choice not in ['1','2','3','4']:
            print('Invalid!'); continue

        try:
            a = float(input('First:  '))
            b = float(input('Second: '))
        except ValueError:
            print('Enter valid numbers!'); continue

        ops = {'1':a+b,'2':a-b,'3':a*b}
        if choice == '4':
            if b == 0: print('Cannot divide by zero!')
            else: print(f'Result: {a/b:.4f}')
        else:
```

# Course Complete!

## أتممت الكورس بنجاح!

**What's Next? | ماذا بعد؟**

- → Odoo Development Fundamentals | أساسيات تطوير أودو
- → File I/O & Libraries (os, json, datetime) | الملفات والمكتبات
- → Odoo ORM & Module Building | أودو وبناء الموديولات ORM